Pygame Zero Basics Cheat Sheet

CS4040 · Unit 3

Key terms

- **`screen`**: the object for the game window you draw on.
- **`Actor`**: a built-in class for a picture you draw and move on the screen.
- **tuple**: values in parentheses that can't be changed one at a time, like `(x, y)` positions and RGB colors.
- Pygame Zero docs · color names · text options

Assets

Folder	Holds	File types
`images`	pictures for Actors	`.png`
`sounds`	sound effects	`.wav` or `.ogg`
`music`	background music	`.ogg` or `.mp3`

Asset names follow the same rules as Python variables: all lowercase, no spaces, only letters, numbers and underscores, and they must begin with a letter. Refer to an asset by its name without the extension: `"alien"`, not `"alien.png"`.

Program structure

Every Pygame Zero program starts with two import lines and ends with `pgzrun.go()`.

```
# program must start with these two lines
import pgzrun
from pgzero.builtins import *

...

pgzrun.go()  # program must always end with this
```

```
# Pygame Zero constants
WIDTH = 712
HEIGHT = 508
TITLE = "Demo Game"
```

- **`start()`**: called once when the game starts, before the game loop begins. Set up the game's starting state here.
- **`update()`**: called repeatedly (about 60 times per second) to change the game's state: move characters, check conditions, change variables.
- **`draw()`**: called repeatedly to draw the current state of the game: backgrounds, shapes and characters.

Colors

Write a color as	Example	Meaning
a color name string	`"darkturquoise"`	any name from the Pygame Zero color list
an RGB tuple	`(150, 0, 100)`	red = 150, green = 0, blue = 100, each from 0 to 255

Screen

The origin `(0, 0)` is the **top-left** corner of the window. `x` increases to the right and `y` increases **down**, so the bottom-right corner is `(WIDTH, HEIGHT)`.

```
screen.clear()  # removes everything from the window
screen.fill("darkslateblue")  # fill with a color name
screen.fill((0, 0, 0))  # or with an RGB tuple
```

```
# screen.draw.rect((x, y), (width, height), "color")
screen.draw.rect((0, 0), (100, 200), "blue")  # outlined rectangle
# screen.draw.filled_rect((x, y), (width, height), "color")
screen.draw.filled_rect((25, 50), (50, 80), "red")  # filled rectangle
```

```
# screen.draw.line((x1, y1), (x2, y2), "color")
screen.draw.line((0, 0), (100, 100), "purple")
```

```
# screen.draw.circle((x, y), radius, "color")
screen.draw.circle((250, 150), 60, "purple")  # outlined circle
# screen.draw.filled_circle((x, y), radius, "color")
screen.draw.filled_circle((120, 120), 40, "red")  # filled circle
```

```
screen.draw.text("hello", (20, 200))  # top-left corner at (20, 200)
screen.draw.text("hello", (100, 100), fontsize=32)
screen.draw.text("hello", (0, 100), fontsize=32, color="orange")
screen.draw.text("hello world", centery=50, right=300)
screen.draw.text("BARK!", midtop=(400, 0), color="red")
```

Actors

```
robot = Actor("robot_idle")  # image: images/robot_idle.png
```

```
robot.pos = (WIDTH // 2, HEIGHT // 2)  # center of the robot
```

```
if robot.image == "robot_idle":
    robot.image = "robot_jump"
elif robot.image == "robot_jump":
    robot.image = "robot_idle"
```

```
robot.scale = 2  # twice as large (1 = original, 0.5 = half)
robot.angle = 45  # rotate 45 degrees counterclockwise
robot.flip_h = True  # flip horizontally (also flip_v)
robot.flip_h = False  # turn the flip back off
```

Actor positions

```
# one coordinate, an int: get or set
alien.x  # center x
alien.y  # center y
alien.top  # top edge y (smallest y)
alien.bottom  # bottom edge y (largest y)
alien.left  # left edge x (smallest x)
alien.right  # right edge x (largest x)
alien.width  # width in pixels (read only)
alien.height  # height in pixels (read only)
```

```
# a point, an (x, y) tuple: get or set
alien.pos = (100, 50)  # center
alien.topleft = (10, 10)  # top-left corner
alien.midright = (20, 20)  # middle of the right edge
alien.midtop  # also midbottom, topright, bottomleft...
```

```
alien.x += 1  # move right 1 pixel
alien.x -= 1  # move left 1 pixel
alien.y += 1  # move down 1 pixel
alien.y -= 1  # move up 1 pixel
```

Tuples

A **tuple** stores several values in parentheses, separated by commas. Positions and colors are tuples. You read one value with square brackets and an index starting at 0, but you cannot change one value: you replace the whole tuple.

```
new_pos = (300, 400)
new_pos[0]  # 300, the x-coordinate
new_pos[1]  # 400, the y-coordinate
new_pos[0] = 100  # ERROR! Cannot modify a tuple
new_pos = (100, 400)  # correct: assign an entirely new tuple
```

Global and Actor variables

The `global` keyword tells Python that a variable inside a function refers to the global variable defined outside it.

You do **not** need `global` when you are…	You **do** need `global` when you are…
only *reading* the variable	*reassigning* the variable to a new value
changing something *inside* it (an Actor's position, appending to a list)	changing an `int`, `float`, `bool`, `str` or tuple

A global frame counter (`frame = 0` at the top of the file):

```
def update():
    global frame  # we reassign frame, so we need global
    frame += 1
    if frame % 10 == 0:  # every 10 frames
        alien.x += 1
```

Actor variables attach your own values to one Actor, no `global` needed:

```
def start():
    alien.pos = (0, 500)  # pos is built in
    alien.velocity_x = 1  # velocity_x is ours
    alien.velocity_y = -1  # velocity_y is ours
```

```
def update():
    alien.x += alien.velocity_x
    alien.y += alien.velocity_y
```